Heart Disease Risk Prediction – End-to-End MLOps Pipeline

Course: AIMLCZG523 Machine Learning Operations – Assignment 01

Student: Bala Gangadhara Tilak · **BITS ID:** 2025AE05975

Repository: https://github.com/TilakMtech/heart-mlops

Demo video: 3rd_sem_MLOps - Google Drive

Date: October 2026

1. Introduction and objective

I built this project to take a heart-disease classifier through the full MLOps workflow: prepare the data, compare models, track experiments, package the selected model, and serve predictions through an API. The aim is to make each step repeatable and to check model quality before a new version moves forward.

The project uses 303 Cleveland records and 13 input measurements. Cleaning and feature processing live in `src/heart`, so training, notebooks, tests, and serving share the same implementation. Docker, Kubernetes, and monitoring complete the deployment workflow shown in Figure 1. This is a learning project on a small dataset; the results do not establish clinical readiness.

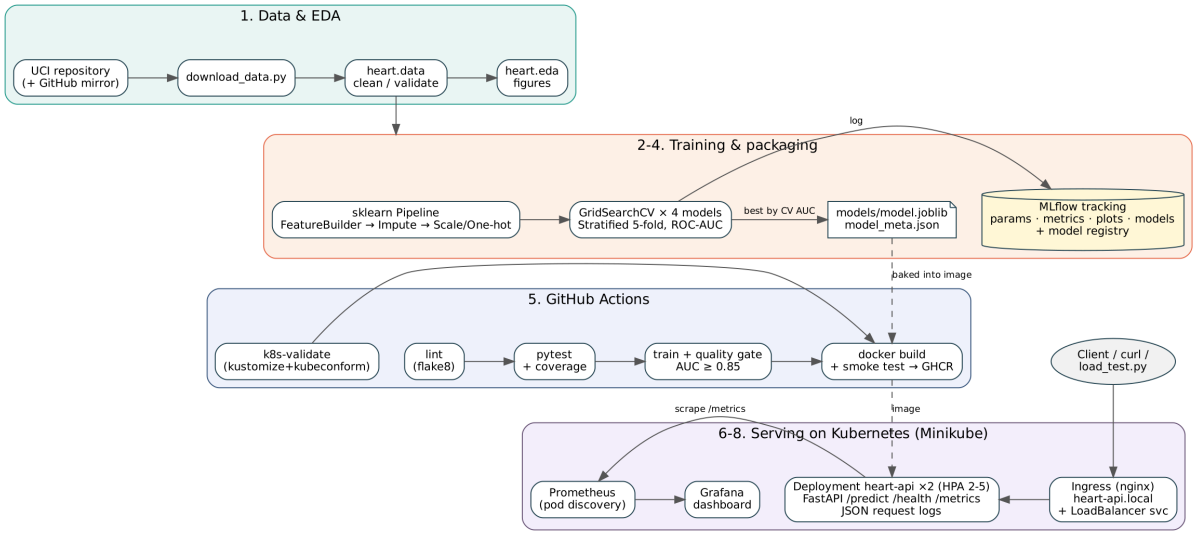


Figure 1 – Solution architecture

2. Setup and installation

Step	Command
Create environment	`python3 -m venv .venv && source .venv/bin/activate` (Python 3.11 or 3.12)
Install	`pip install -r requirements-dev.txt` (or `conda env create -f environment.yml`)
Data	`make data`
EDA	`make eda`
Train + MLflow	`make train`, then `make mlflow` (UI on port 5000)
Tests / lint	`make test`, `make lint`
Container	`make docker-build docker-run smoke`
Monitoring stack	`docker compose up -d --build`
Kubernetes	`./scripts/deploy_minikube.sh`

Dependencies are pinned in three layers: `requirements-serve.txt` (only what the API needs — keeps the image small), `requirements.txt` (adds training, plotting and MLflow) and `requirements-dev.txt` (adds pytest, flake8, Jupyter).

Execution environment. The pipeline was executed end-to-end in the BITS lab: Linux (x86_64, 4 vCPU, 15 GB RAM), Python 3.12.9, Docker 28.0.1, Minikube v1.35.0 (docker driver), kubectl v1.31.1. The Docker image and the GitHub Actions jobs use Python 3.11; the code and pinned dependencies are tested on both 3.11 and 3.12.

3. Data acquisition and EDA

3.1 Acquisition

`scripts/download_data.py` downloads `processed.cleveland.data` from the UCI repository. Because UCI is sometimes blocked by corporate proxies, the script falls back to a GitHub mirror of the same 303 records and maps its text labels back to the UCI numeric codes, so the output schema is identical either way. The result is `data/raw/heart_raw.csv` with 14 columns.

3.2 Cleaning

Cleaning (`heart.data.clean`) is intentionally stateless:

- `?` markers become NaN (4 in `ca`, 2 in `thal`);
- the target `num` (0–4) is binarised to `target = num > 0`;
- out-of-range values (e.g. cholesterol = 0) and unknown category codes become NaN;
- exact duplicates are removed (none found).

Imputation is deferred to the model pipeline (median for continuous, mode for categorical) so that it is learned on training folds only and travels with the model — this avoids leakage and guarantees the API treats missing `ca`/`thal` exactly as training did. The cleaned file is `data/processed/heart_clean.csv` (303 rows).

3.3 Exploratory analysis

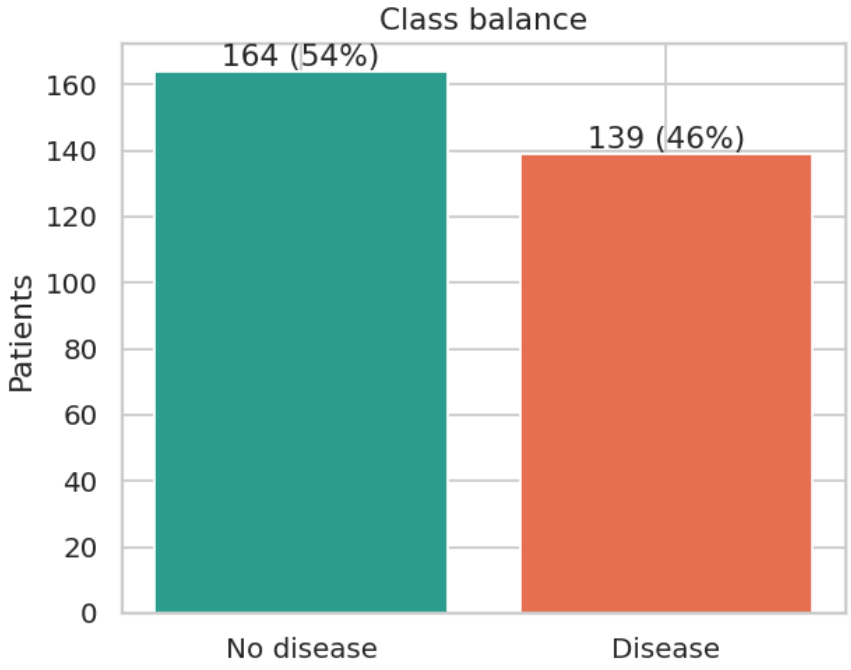


Figure 2 – Class balance

The classes are close to balanced (164 without disease, 139 with disease; 45.9 % positive), so accuracy is meaningful, but ROC-AUC is used for model selection because it is threshold-free.

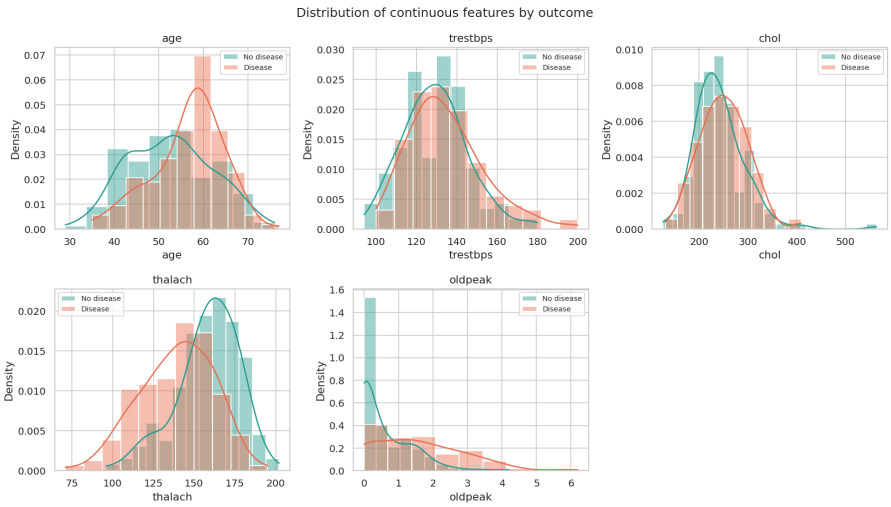


Figure 3 – Continuous features by outcome

Patients with disease reach a lower maximum heart rate (`thalach`) and show larger exercise ST depression (`oldpeak`). `chol` and `trestbps` overlap heavily between classes.

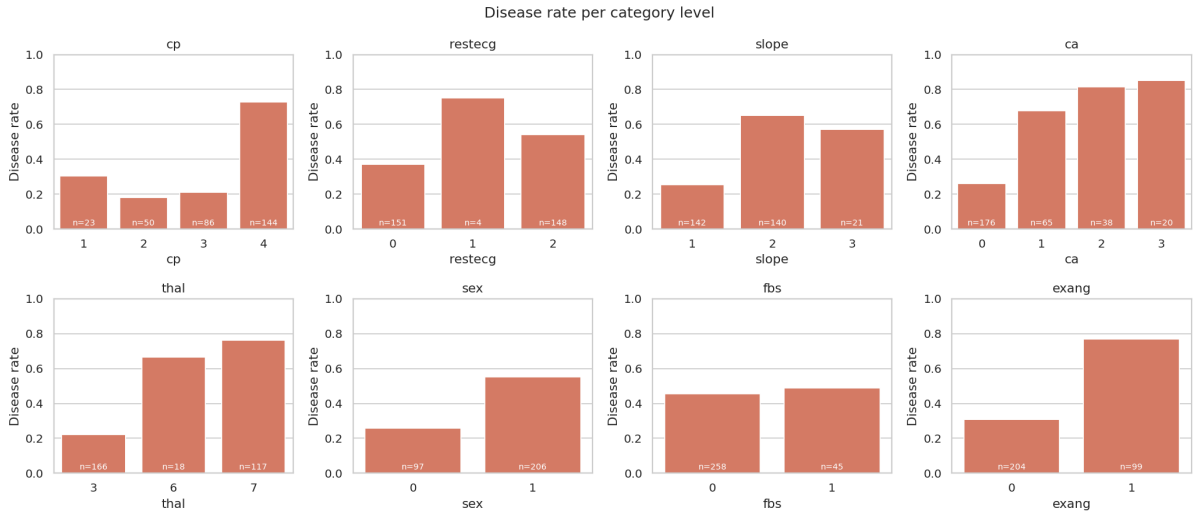


Figure 4 – Disease rate per category

The plotted disease rate is above 70% for `cp = 4`, `thal = 7`, `ca = 2` or `3`, and exercise angina; `ca = 1` is below 70%. `restecg = 1` has only 4 patients, so its high rate is not reliable — I would avoid drawing a strong conclusion from that category. One-hot encoding represents the codes without imposing a numeric ordering; it does not solve the small-sample limitation.

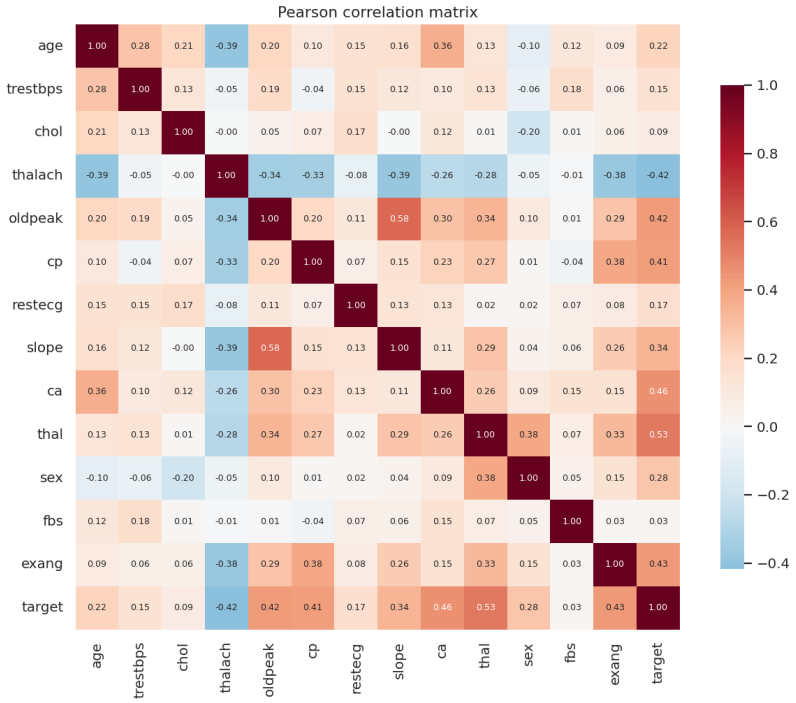


Figure 5 – Correlation heatmap

The strongest absolute correlations with the target are `thal` (0.53), `ca` (0.46), `exang` (0.43), `oldpeak` (0.43), `thalach` (0.42) and `cp` (0.41). No pair of predictors is so strongly correlated that it would need to be dropped.

4. Feature engineering and model development

4.1 Features

The preprocessing is a single scikit-learn `Pipeline`:

1. `HeartFeatureBuilder` (custom transformer) adds three derived features: `hr_reserve_pct = thalach / (220 − age)` (share of age-predicted maximum heart rate reached), `age_oldpeak = age * oldpeak` (an experimental age/ST-depression interaction), and `st_risk_flag = 1 if oldpeak ≥ 2 or exang = 1`.
2. `ColumnTransformer`: continuous → median imputation + `StandardScaler`; `cp`, `restecg`, `slope`, `ca`, `thal` → mode imputation + one-hot; `sex`, `fbs`, `exang` → mode imputation (already 0/1).
3. Estimator.

4.2 Candidate models and tuning

Candidate	Search space	Purpose
logreg_baseline	C ∈ {0.01…10}, L1/L2; **no engineered features**	Ablation
logreg	C ∈ {0.01…3}, L1/L2, class_weight ∈ {None, balanced} (48 combos)	Interpretable linear model
random_forest	n_estimators, max_depth, min_samples_leaf, max_features (36 combos)	Non-linear bagging
gradient_boosting	n_estimators, learning_rate, max_depth, subsample (16 combos)	Non-linear boosting

Data is split 80/20 (stratified, seed 42). Each candidate is tuned with `GridSearchCV` on the training split using stratified 5-fold CV and ROC-AUC. The best configuration is then cross-validated again for accuracy, precision,

recall, F1 and ROC-AUC (mean ± std), and finally scored on the 61-patient hold-out set at the fixed 0.4 threshold, with 0.5 results retained for comparison. **Selection rule:** highest mean CV ROC-AUC; the hold-out set is never used for selection, even as a tie-breaker. Exact ties keep candidate order. The same folds are reused after tuning, so these CV scores can be optimistic.

4.3 Results

Model	CV accuracy	CV precision	CV recall	CV ROC-AUC	Test accuracy	Test recall	Test ROC-AUC
logreg_baseline	0.843 ± 0.037	0.824	0.846	0.904 ± 0.015	0.885	0.964	0.969
logreg	0.810 ± 0.035	0.763	0.855	0.905 ± 0.013	0.869	0.964	0.970
random_forest	0.781 ± 0.048	0.741	0.810	0.891 ± 0.026	0.853	0.964	0.965
gradient_boosting	0.789 ± 0.022	0.762	0.810	0.883 ± 0.026	0.836	0.929	0.958

All label-based metrics above use threshold **0.4**; ROC-AUC uses probabilities.

Selected hyper-parameters: 'C: 0.3', 'class_weight: balanced', 'penalty: l1'.

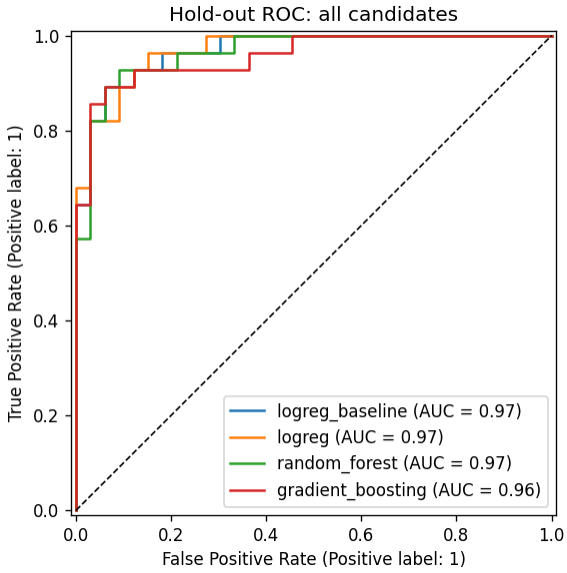


Figure 6 – Hold-out ROC curves for all candidates

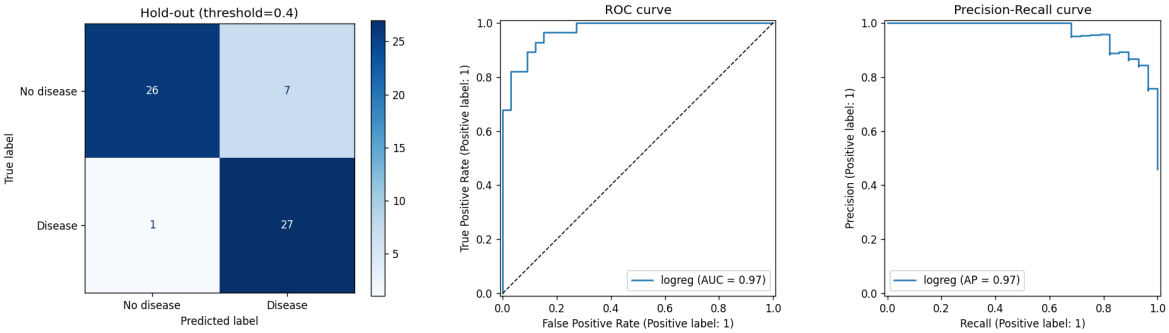


Figure 7 – Selected model: confusion matrix, ROC and PR curves

Discussion. I would keep `logreg` because it is the winner under the training CV selection rule, with mean ROC-AUC 0.9052. The raw-feature logistic model scores 0.9045, compared with 0.9052 for the version with engineered features. I do not take a small difference on this dataset as proof that the extra features help. The age × ST-depression feature is an experiment, and the baseline remains a useful check on whether that extra complexity is worthwhile. The exported model gives `age_oldpeak` a zero coefficient under L1 regularisation, so I cannot claim that the new interaction improved the selected model.

I fixed the threshold at 0.4 before evaluation to favour recall. For the selected model, mean CV recall changes from 0.7739 at 0.5 to 0.8553 at 0.4, while precision changes from 0.8696 to 0.7631. On the hold-out set, recall is 0.9643 and precision is 0.7941. The comparison CSV retains both thresholds, the confusion matrices use 0.4, and the API reads 0.4 from metadata. This is a project-level trade-off, not a clinically validated operating point.

There are only 61 hold-out records, and the same CV folds were used to tune and score each candidate. I would not claim statistical equivalence or clinical performance from these results. Nested CV and a fresh external dataset would give a stronger check, especially because this project's hold-out results have already been reviewed during development.

5. Experiment tracking (MLflow)

Every execution of `python -m heart.train` creates one parent run (`session-<timestamp>`) with dataset size, split and seed parameters and the cleaned dataset as an artifact. Each candidate is a nested child run that logs:

- **params:** best hyper-parameters, grid size, whether engineered features were used;
- **metrics:** `cv_*_mean`/`cv_*_std` for five metrics, `test_*` hold-out metrics, search time;
- **artifacts:** full grid-search results CSV, evaluation plot, feature-importance plot, and the fitted pipeline as an MLflow model with an inferred signature and an input example.

The parent run additionally stores the model comparison table, the ROC overlay and the exported `model_meta.json`, and tags the selected model. The winner is registered as `heart-disease-classifier` in the MLflow model registry.

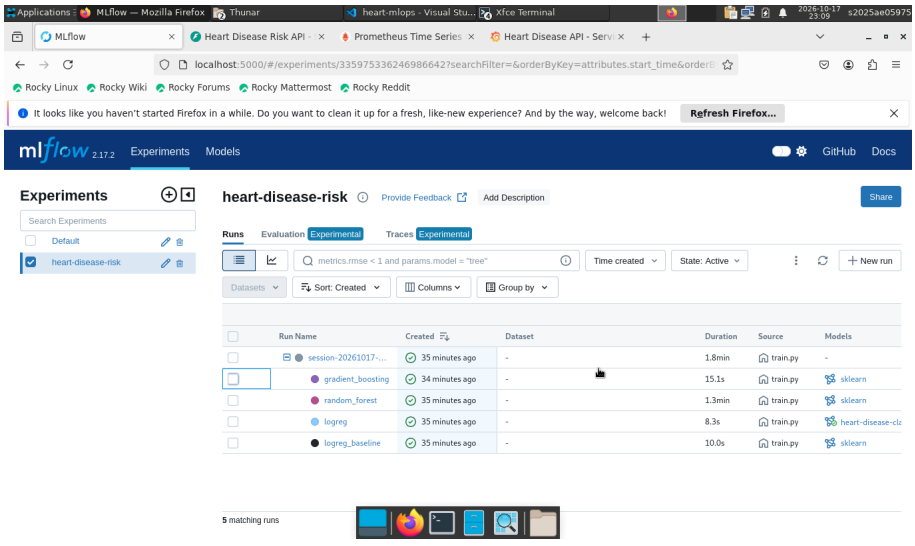


Figure 8 – MLflow experiment with parent and nested runs

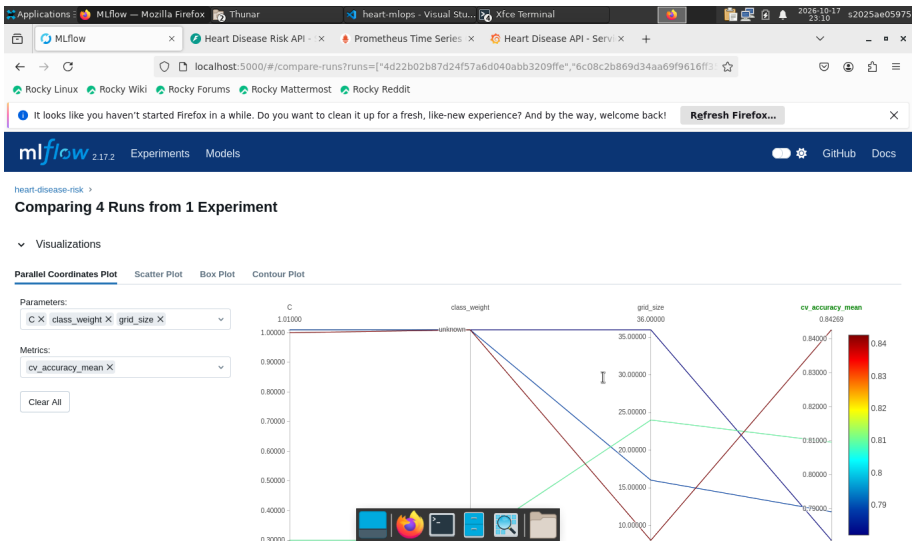


Figure 9 – MLflow run comparison

6. Model packaging and reproducibility

- The complete pipeline (feature builder, imputers, scaler, encoder and classifier) is saved as one object in `models/model.joblib`; the same model is also stored in MLflow format.
- `models/model_meta.json` records model name, version, MLflow run IDs, hyper-parameters, engineered features, quality-gate limits, metrics, the input feature order, decision threshold and the scikit-learn version.
- All random seeds are fixed (`RANDOM_STATE = 42`), library versions are pinned, and every stage runs from a clean environment with `make install data train`.
- `heart.predict.HeartRiskModel` is the only inference code path; the notebooks, CLI and API all use it.

7. CI/CD pipeline and automated testing

7.1 Tests

The `tests/` folder contains 48 pytest tests that need no network or real dataset (they use a synthetic fixture):

- **data:** target binarisation, `?` handling, out-of-range nulling, duplicates, schema errors;
- **features:** derived-feature formulas, no NaNs after preprocessing, scaling, unseen categories;
- **model:** pipeline fit/predict for LR and RF, learnability check, candidate registry, inference wrapper;
- **API:** health, prediction contract, optional fields, validation errors (422), batch, metrics, 503 without a model.

7.2 GitHub Actions workflow

Job	Steps	Artifacts
lint	flake8	`flake8-report.txt`
k8s-validate	kustomize build, kubeconform (k8s 1.31 schemas)	rendered manifests
test	pytest + coverage	JUnit XML, coverage XML
train	download → EDA → train → quality gate (CV AUC ≥ 0.85 and mean CV recall ≥ 0.80 at threshold 0.4) → job summary	model, MLflow store, reports, logs
docker	build image with new model → run → smoke test → push to GHCR (main)	container logs

Jobs are chained with `needs`, so a lint error, a failing test, a model below the quality gate or a failing smoke test stops the pipeline with a red status and clear logs.

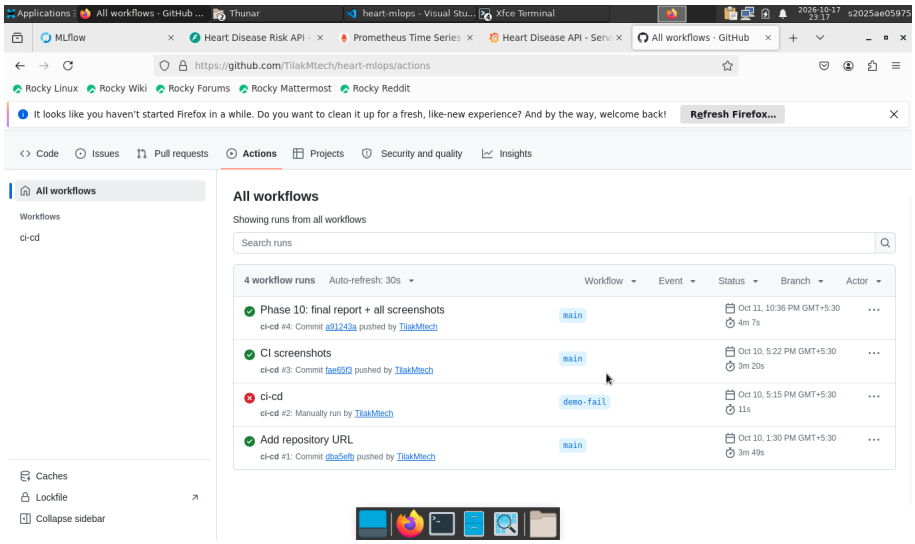


Figure 10 – Successful GitHub Actions run

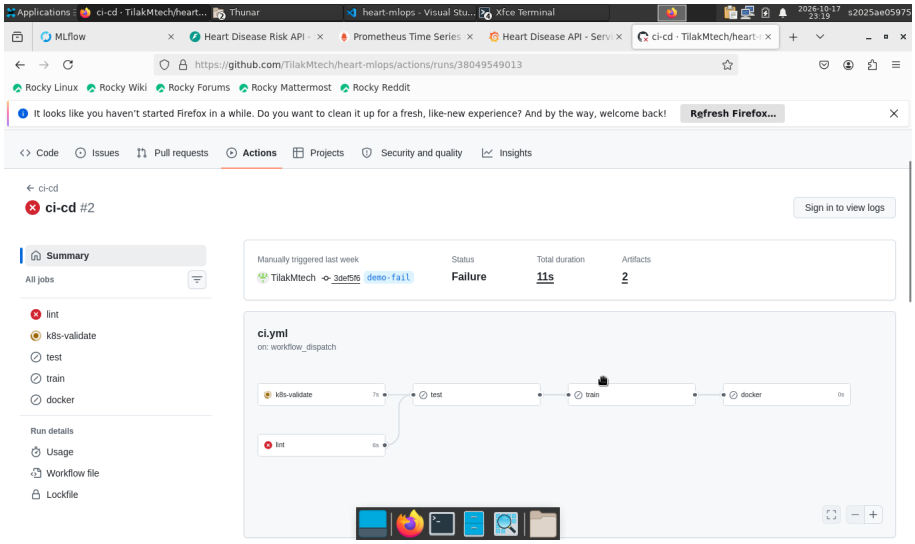


Figure 11 – Failing run with clear logs

8. Model containerisation

The `Dockerfile` uses `python:3.11-slim`, installs only `requirements-serve.txt`, copies the package, API and model, runs as a non-root user (UID 1001) and defines a `HEALTHCHECK` on `/health`. One Uvicorn worker runs per container so Prometheus counters stay consistent; scaling is done with Kubernetes replicas.

API contract:

Endpoint	Purpose
`POST /predict`	One patient JSON → `prediction`, `label`, `probability_disease`, `confidence`, `risk_band`, `model_version`, `request_id`
`POST /predict/batch`	Up to 500 records
`GET /health`	200 when the model is loaded, 503 otherwise
`GET /metrics`	Prometheus exposition
`GET /docs`	Swagger UI

Inputs are validated by pydantic (ranges and allowed codes); invalid input returns 422. `ca` and `thal` are optional and imputed by the pipeline. `scripts/smoke_test.sh` verifies all of this against a running container.

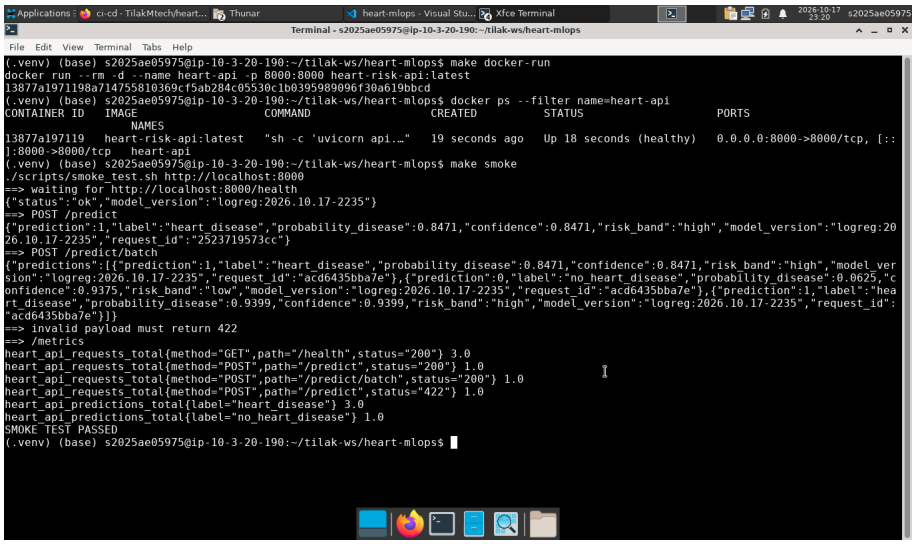


Figure 12 – Docker build, run and smoke test

9. Production deployment (Kubernetes / Minikube)

Manifests in `k8s/` are applied with Kustomize (`kubectl apply -k k8s/`):

- **Namespace** `heart-mlops`;
- **Deployment** – 2 replicas, rolling update with `maxUnavailable: 0`, readiness and liveness probes on `/health`, CPU/memory requests and limits, non-root security context, Prometheus scrape annotations;
- **Service** of type `LoadBalancer` (external IP through `minikube tunnel`);
- **Ingress** (NGINX) on host `heart-api.local`;
- **HorizontalPodAutoscaler** 2–5 replicas at 70 % CPU.

`scripts/deploy_minikube.sh` performs the whole flow: start Minikube, enable add-ons, build and load the image, apply manifests, wait for rollout and run the smoke test against the service URL.

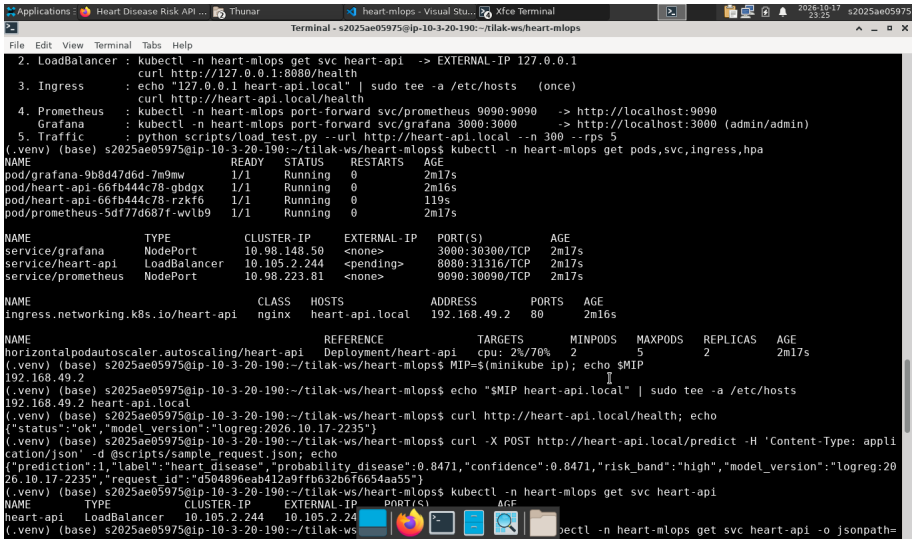


Figure 13 – Kubernetes resources

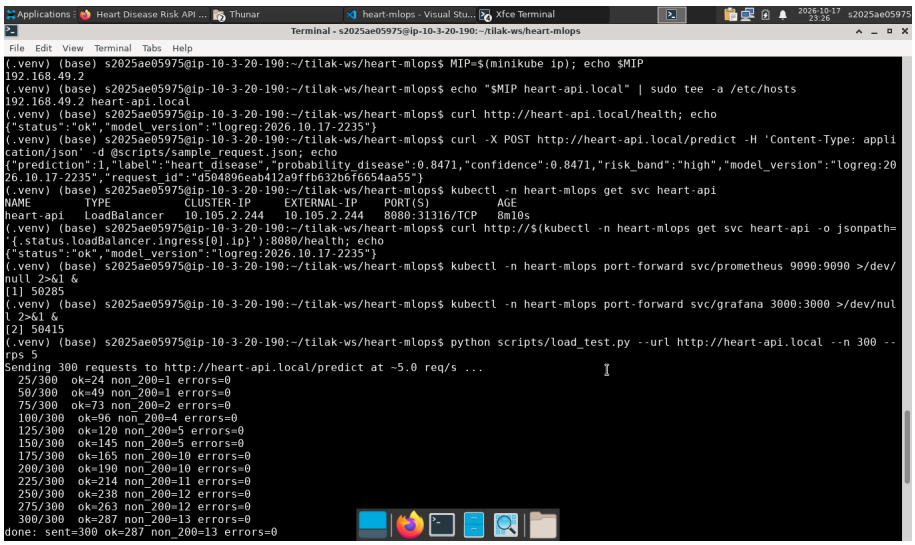


Figure 14 – Prediction through the Ingress

10. Monitoring and logging

- **Logging:** a middleware writes one JSON line per request (request ID, method, path, status, latency, client) and one per prediction (inputs, probability, label). The request ID is returned in the `x-request-id` header so a client can trace its call in `kubectl logs`.
- **Metrics:** `heart_api_requests_total{method,path,status}`, `heart_api_request_latency_seconds` (histogram), `heart_api_predictions_total{label}`, `heart_api_disease_probability` (histogram) and `heart_api_model_info{version}`.
- **Prometheus** discovers the API pods through their annotations (pod service discovery with RBAC); for local use, `docker-compose.yml` runs the same stack.
- **Grafana** is provisioned automatically with a dashboard showing total predictions, request rate, p95 latency, error rate, latency percentiles, predictions per class, and the mean predicted probability as a simple drift signal — a sustained shift in that line means the incoming patient mix has changed and the model should be reviewed.

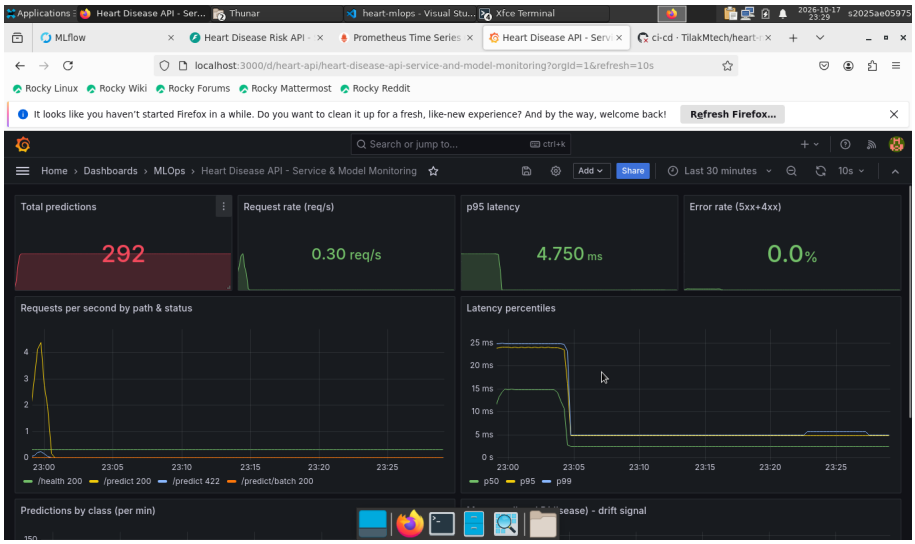


Figure 15 – Grafana dashboard under load

11. Conclusions and future work

I now have a shared pipeline that prepares the data, compares four models, tracks runs in MLflow, and exports a model for the API. In this run, `logreg` achieved 0.9052 mean CV ROC-AUC and 0.8553 mean CV recall at threshold 0.4. Hold-out recall was 0.9643. These numbers describe this experiment, rather than expected performance on new patient populations.

The three changes I made are an age × ST-depression feature in place of cholesterol-per-age, a fixed recall-oriented threshold used in both evaluation and serving, and a recall gate alongside the existing AUC gate in CI. The feature is still experimental, and lower thresholds can increase false positives. I would next check probability calibration, validate the model and threshold on new data, and add data-drift checks and a model-promotion workflow. The existing deployment and monitoring setup provides a place to apply those checks; it does not remove the need for validation.

The existing Word report and deployment screenshots predate this revision; `REPORT.md`, the executed notebooks, and the new training artifacts are the updated evidence.

12. Links

- Code repository: https://github.com/TilakMtech/heart-mlops
- API access: see README §6–7 (local Docker on port 8000, or Minikube service / Ingress `heart-api.local`)

Demo video: 3rd_sem_MLOps - Google Drive